

P1144R4

Object relocation in terms of move plus destroy

Published Proposal, 2019-06-17

Issue Tracking:

[Inline In Spec](#)

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1144-object-relocation.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1144-object-relocation.html

Abstract

We define a new verb, "relocate," which is equivalent to a move and a destroy (analogous to the existing verb "swap," which is equivalent to a move, two move-assignments, and a destroy). For many C++ types, the "relocate" operation is implementable as a single `memcpy`. We provide a standard trait to detect types which are trivially relocatable, for the benefit of library writers. Finally, we provide a portable way for a user-defined type (e.g. `boost::shared_ptr`) to warrant to the implementation that it is trivially relocatable.

Table of Contents

1 Changelog

2 Introduction and motivation

2.1 Optimizations enabled by trivial relocatability

2.1.1 Vector resize

2.1.2 Swap

2.1.3 More efficient small-buffer type-erasure

2.1.4 More efficient fixed-capacity containers

2.1.5 Assertions, not assumptions

2.2 The most important benefit

3 Design goals

3.1 Standard library types such as `std::string`

3.2 Program-defined types that follow the Rule of Zero

3.3 Program-defined types with non-defaulted special members

4 Proposed language and library features

4.1 Why `[[trivially_relocatable(bool)]]`?

5 Proposed wording for C++2b

5.1 Relocation operation

5.2 Algorithm `relocate_at`

5.3 Algorithm `uninitialized_relocate`

5.4 Algorithm `uninitialized_relocate_n`

5.5 Trivially relocatable type

5.6 `[[trivially_relocatable]]` attribute

5.7 Type traits `is_relocatable` etc.

5.8 `Relocatable` concept

6 Rationale and alternatives

6.1 Why not destructive move?

6.2 Attribute `[[maybe_trivially_relocatable]]`

6.3 Should `relocate_at` be a customization point?

6.4 Unintuitive `is_nothrow_relocatable`

7 Acknowledgements

Appendix A: Straw polls

Polls taken of the Internet between 2018-11-12 and 2018-11-21

Poll taken of EWGI at San Diego on 2018-11-07

Polls taken of SG14 at CppCon on 2018-09-26

Appendix B: Sample code

Reference implementation of `relocate_at` and `uninitialized_relocate`

Conditionally trivial relocation

Appendix C: Examples of non-trivially relocatable class types

Class contains pointer to self

Allocated memory contains pointer to self

Class invariant depends on `this`

Program invariant depends on `this`

Appendix D: Implementation experience

Index

Terms defined by this specification

References

Normative References

Informative References

Issues Index

§ 1. Changelog

- R3 (post-Kona):
 - User-provided copy/move assignment operators inhibit "natural" trivial relocatability, just like user-provided move constructors and destructors. This is intended to grant us permission to optimize `vector::insert`.

- User-provided copy constructors inhibit "natural" trivial relocatability, just like user-provided move constructors and destructors.
- Adopted `[[trivially_relocatable(bool)]]`. The attribute-argument is optional.
- Added `std::relocate_at(source, dest)`. Note that this is *not* a customization point.
- Removed the core-language blanket wording that permitted "the implementation" to eliminate moves and destroys of trivially relocatable types. Instead, the existence of `std::relocate_at` and handwaviness of e.g. `[vector.capacity]` combine to strongly imply (but still not to mandate) that library vendors will use `std::relocate_at` wherever it helps with performance. We no longer propose to permit eliminating moves and destroys in e.g. `return` statements, except as permitted under the as-if rule.

§ 2. Introduction and motivation

C++17 knows the verbs "move," "copy," "destroy," and "swap," where "swap" is a higher-level operation composed of several lower-level operations. To this list we propose to add the verb "relocate," which is a higher-level operation composed of exactly two lower-level operations. Given an object type `T` and memory addresses `src` and `dst`, the phrase "**relocate** a `T` from `src` to `dst`" means no more and no less than "*move-construct* `dst` from `src`, and then immediately *destroy* the object at `src`."

Just as the verb "swap" produces the adjective "swappable," the verb "relocate" produces the adjective "relocatable." Any type which is both move-constructible and destructible is **relocatable**. The notion can be modified by adverbs: we say that a type is **nothrow relocatable** if its relocation operation is noexcept, and we say that a type is **trivially relocatable** if its relocation operation is trivial (which, just like trivial move-construction and trivial copy-construction, means "the operation is tantamount to a `memcpy`").

Almost all relocatable types are trivially relocatable: `std::unique_ptr<int>`, `std::vector<int>`, `std::string`. Non-trivially relocatable types exist but are rare; see [Appendix C: Examples of non-trivially relocatable class types](#).

§ 2.1. Optimizations enabled by trivial relocatability

§ 2.1.1. Vector resize

If we have a reliable way of detecting "trivial relocatability," we can optimize any routine that performs the moral equivalent of `realloc`, including

```
std::vector<R>::resize  
std::vector<R>::reserve  
std::vector<R>::emplace_back  
std::vector<R>::push_back
```

[[Bench](#)] (presented at C++Now 2018) shows a 3x speedup on `std::vector<std::unique_ptr<int>>::resize`. [This Reddit thread](#) demonstrates a similar 3x speedup using the online tool Quick-Bench.

As observed in [[CppChat](#)] (@21:55): Just as with C++11 move semantics, you can write benchmarks to show whatever speedup you like. The more complicated your types' move-constructors and destructors, the more time you save by eliminating calls to them.

§ 2.1.2. Swap

Given a reliable way of detecting trivial relocatability, we can optimize any routine that uses the moral equivalent of `std::swap`, such as

```
std::swap  
std::sort  
std::vector<R>::insert
```

§ 2.1.3. More efficient small-buffer type-erasure

Given a reliable way of detecting trivial relocatability, we can de-duplicate the code generated by small-buffer-optimized (SBO) type-erasing wrappers such as `std::function` and `std::any`. For these types, a *move* of the wrapper object is implemented in terms of a *relocation* of the contained object. (See for example [libc++'s std::any](#), where the function that performs the relocation operation is confusingly named `__move`.) In general, the *relocate* operation for a contained type `C` must be uniquely codegenned for each different `C`, leading to code bloat. But a single instantiation suffices to relocate every *trivially* relocatable `C` in the program. A smaller number of instantiations means faster compile times, a smaller text section, and "hotter" code (because a relatively higher proportion of your code now fits in icache).

§ 2.1.4. More efficient fixed-capacity containers

Given a reliable way of detecting trivial relocatability, we can optimize the move-constructor of `fixed_capacity_vector<R,N>`, which can be implemented naïvely as an element-by-element *move* (leaving the source vector's elements in their moved-from state), or can be implemented efficiently as an element-by-element *relocate* (leaving the source vector empty).

For a detailed analysis of this case, see [\[FixedCapacityVector\]](#).

Note: `boost::container::static_vector<R,N>` currently implements the naïve element-by-element-move strategy.

§ 2.1.5. Assertions, not assumptions

Some concurrent data structures might reasonably assert the trivial relocatability of their elements, just as they sometimes assert the stronger property of trivial *copyability* today.

§ 2.2. The most important benefit

Many real-world codebases already contain templates which require trivial relocatability of their template parameters, but currently have no way to *verify* trivial relocatability. For example, [\[Folly\]](#) requires the programmer to warrant the trivial relocatability of any type stored in a

`folly::fbvector`:

```
class Widget {
    std::vector<int> lst_;
};

folly::fbvector<Widget> vec; // FAILS AT COMPILE TIME for lack of warrant
```

But this merely encourages the programmer to add the warrant and continue. An incorrect warrant will be discovered only at runtime, via undefined behavior. (See [Allocated memory contains pointer to self](#) and [\[FollyIssue889\]](#).)

```

class Gadget {
    std::list<int> lst_;
};
// sigh, add the warrant on autopilot
template<> struct folly::IsRelocatable<Gadget> : std::true_type {};

folly::fbvector<Gadget> vec; // CRASHES AT RUNTIME due to fraudulent v

```

If this proposal is adopted, then Folly can start using

`static_assert(std::is_trivially_relocatable_v<T>)` in the implementation of `fbvector`, and the programmer can stop writing explicit warrants. Finally, the programmer can start writing assertions of correctness, which aids maintainability and can even find real bugs. [Example:](#)

```

class Widget {
    std::vector<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Widget>); // correctly S

class Gadget {
    std::list<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Gadget>); // correctly E

```

The improvement in user experience for real-world codebases (such as [Folly](#), [EASTL](#), BDE, Qt, etc.) is the most important benefit to be gained by this proposal.

§ 3. Design goals

Every C++ type already *is* or *is not* trivially relocatable. This proposal does not require any library vendor to make any library type trivially relocatable. (We assume that quality implementations will do so on their own.)

The optimizations discussed above are purely in the domain of library writers. If you're writing a vector, and you detect that your element type `T` is trivially relocatable, then whether you do any special optimization in that case is up to you. This proposal does not require any library vendor to guarantee that any particular optimization happens. (But we assume that quality implementations will do so on their own.)

What C++ lacks is a standard way for library writers to *detect* the (existing) trivial relocatability of a type `T`, so that they can reliably apply their (existing) optimizations. All we really need is to add *detection*, and then all the optimizations described above will naturally emerge without any further special effort by WG21.

There are three kinds of object types that we want to make sure are correctly detected as trivially relocatable. These three cases are important for improving the performance of the standard library, and for improving the correctness of programs using libraries such as [Folly](#)'s `fbvector`.

§ 3.1. Standard library types such as `std::string`

In order to optimize `std::vector<std::string>::resize`, we must come up with a way to achieve

```
#include <string>
static_assert(is_trivially_relocatable< std::string >::value);
```

This could be done unilaterally by the library vendor — via a non-standard attribute (`[[clang::trivially_relocatable]]`), or a member typedef with a reserved name, or simply a vendor-provided specialization of `std::is_trivially_relocatable<std::string>`.

That is, we can in principle solve §2.1 while confining our "magic" to the headers of the implementation itself. The programmer doesn't have to learn anything new, so far.

§ 3.2. Program-defined types that follow the Rule of Zero

In order to optimize the SBO `std::function` in any meaningful sense, we must come up with a way to achieve

```
#include <string>
auto lam2 = [x=std::string("hello")]{};
static_assert(is_trivially_relocatable< decltype(lam2) >::value);
```

Lambdas are not a special case in C++; they are simply class types with all their special members defaulted. Therefore, presumably we should be able to use the same solution for lambdas as for


```
#include <string>
struct A {
    std::string s;
};
static_assert(is_trivially_relocatable< A >::value);
```

Here `struct A` follows the Rule of Zero: its move-constructor and destructor are both defaulted. If they were also *trivial*, then we'd be done. In fact they are non-trivial; and yet, because the type's bases and members are all of trivially relocatable types, the type as a whole *is* trivially relocatable.

§2.2 asks specifically that we make the `static_assert` succeed without breaking the "Rule of Zero." We do not want to require the programmer to annotate `struct A` with a special attribute, or a special member typedef, or anything like that. We want it to Just Work. Even for lambda types. This is a much harder problem than §2.1; it requires standard support in the core language. But it still does not require any new *syntax*.

§ 3.3. Program-defined types with non-defaulted special members

In order to optimize `std::vector<boost::shared_ptr<T>>::resize`, we must come up with a way to achieve

```
struct B {
    B(B&&); // non-trivial
    ~B(); // non-trivial
};
static_assert(is_trivially_relocatable< B >::value);
```

via some standard annotation applied to class type `B` (which in this example is standing in for `boost::shared_ptr`).

Note: We cannot possibly do it without annotation, because there exist examples of types that look just like `B` and *are* trivially relocatable (for example, [libstdc++'s `std::function`](#)) and there exist types that look just like `B` and *are not* trivially relocatable (for example, [libc++'s `std::function`](#)). The compiler cannot "crack open" the definitions of `B(B&&)` and `~B()` to see if they combine to form a trivial operation: the definitions of `B(B&&)` and `~B()` might not even be available in the current translation unit. So, without some kind of opt-in annotation, we cannot achieve our goal.

This use-case is the only one that requires us to design the "opt-in" syntax. In [§2.1](#), any special syntax is hidden inside the implementation's own headers. In [§2.2](#), our design goal is to *avoid* special syntax.

In §2.3, WG21 must actually design user-facing syntax.

Therefore, I believe it would be acceptable to punt on §2.3 and come back to it later. We say, "Sure, that would be nice, but there's no syntax for it. Be glad that it works for core-language and library types. Ask again in three years." As long as we leave the design space open, I believe we wouldn't lose much by delaying a solution to §2.3.

This paper does propose a standard syntax for §2.3 — an attribute — which in turn provides a *simple and portable* way for library vendors to implement §2.1.

§ 4. Proposed language and library features

This paper proposes five separate additions to the C++ Standard. These additions introduce "relocate" as a well-supported C++ notion on par with "swap," and furthermore, successfully communicate trivial relocatability in each of the three use-cases above.

- A new standard algorithm, `uninitialized_relocate(first, last, d_first)`, in the `<memory>` header.
- Additional type traits, `is_relocatable<T>` and `is_nothrow_relocatable<T>`, in the `<type_traits>` header.
- A new type trait, `is_trivially_relocatable<T>`, in the `<type_traits>` header. This is the *detection* mechanism.
- A new core-language rule by which a class type's "trivial relocatability" is inherited according to the Rule of Zero.
- A new attribute, `[[trivially_relocatable]]`, in the core language. This is the *opt-in* mechanism for program-defined types.

These five bullet points are severable to some degree. For example, if the `[[trivially_relocatable]]` attribute (point 5) is adopted, library vendors will certainly use it in their implementations; but if the attribute is rejected, library vendors could still indicate the trivial relocatability of certain standard library types by providing library specializations of `is_trivially_relocatable` (point 3).

Points 1 and 2 are completely severable from points 3, 4, and 5; but we believe these algorithms should be provided for symmetry with the other uninitialized-memory algorithms in the `<memory>` header (e.g. `uninitialized_move`) and the other trios of type-traits in the `<type_traits>` header (e.g. `is_destructible`, `is_nothrow_destructible`, `is_trivially_destructible`). I do

not expect these templates to be frequently useful, but I believe they should be provided, so as not to surprise the programmer by their absence.

Points 3 and 4 together motivate point 5. In order to achieve the goal of [§ 3.2 Program-defined types that follow the Rule of Zero](#), we *must* define a core-language mechanism by which we can "inherit" trivial relocatability. This is especially important for the template case.

```
template<class T>
struct D {
    T t;
};

// class C comes in from outside, already marked, via whatever mechanism
constexpr bool c = is_trivially_relocatable< C >::value;
constexpr bool dc = is_trivially_relocatable< D<C> >::value;
static_assert(dc == c);
```

We strongly believe that `std::is_trivially_relocatable<T>` should be just a plain old class template, exactly like `std::is_trivially_destructible<T>` and so on. The core language *should not know or care* that the class template `is_trivially_relocatable` exists, any more than it knows that the class template `is_trivially_destructible` exists.

We expect that the library vendor will implement `std::is_trivially_relocatable`, just like `std::is_trivially_destructible`, in terms of a non-standard compiler builtin whose natural spelling is `__is_trivially_relocatable(T)`. This builtin has been implemented in Clang; see [\[D50119\]](#). The compiler computes the value of `__is_trivially_relocatable(T)` by inspecting the definition of `T` (and the definitions of its base classes and members, recursively). This recursive process "bottoms out" at primitive types, or at any type with a user-provided move or destroy operation. For safety, classes with user-provided move or destroy operations (e.g. [Appendix C: Examples of non-trivially relocatable class types](#)) must be assumed *not* to be trivially relocatable. To achieve the goal of [§ 3.3 Program-defined types with non-defaulted special members](#), we must provide a way that such a class can "opt in" and warrant to the implementation that it is in fact trivially relocatable (despite being non-trivially move-constructible and/or non-trivially destructible).

In point 5 we propose that the opt-in mechanism should be an attribute. The programmer of a trivially relocatable but non-trivially destructible class `C` will mark it for the compiler using the attribute:

```

struct [[trivially_relocatable]] C {
    C(C&&); // defined elsewhere
    ~C(); // defined elsewhere
};
static_assert(is_trivially_relocatable< C >::value);

```

The attribute overrides the compiler's usual computation.

An example of a "conditionally" trivially relocatable class is shown in [Conditionally trivial relocation](#).

The attribute is severable; WG21 could adopt all the rest of this proposal and leave vendors to implement `[[clang::trivially_relocatable]]`, `[[gnu::trivially_relocatable]]`, etc., as non-standard extension mechanisms. In that case, we would strike §4.5 and one bullet point from §4.4; the rest of this proposal would remain exactly the same.

§ 4.1. Why `[[trivially_relocatable(bool)]]`?

It was suggested by numerous reviewers that `[[trivially_relocatable]]` should take a (perhaps optional) boolean parameter, as in `[[trivially_relocatable(true)]]`. This allows us to write complicated conditions directly inline, instead of using metaprogramming to inherit the right behavior from a conditional base class.

See [Conditionally trivial relocation](#) for an example of how this can be used.

There is no technical obstacle to adding an arbitrary C++ expression as the parameter to an attribute. The grammar for balancing `[]` and `()` in attribute parameters has been there since C++11. There is already prior art for arbitrary expressions in attributes; see [Clang's `\[\[diagnose\_if\(bool\)\]\]` attribute](#). EWG has also previously considered an attribute `[[assume_aligned(expr)]]`; it was rejected in favor of [P1007R3](#)'s non-attribute approach, but not because of the `(expr)` part specifically.

The major downside I see to `[[trivially_relocatable(expr)]]` is that it could lead to an arbitrarily complicated C++ expression appearing in an awkward position. But this is not necessarily worse, and perhaps better, than having to do the metaprogramming tricks shown in [Conditionally trivial relocation](#).

§ 5. Proposed wording for C++2b

The wording in this section is relative to [WG21 draft N4800](#).

§ 5.1. Relocation operation

Add a new section in [\[definitions\]](#):

ISSUE 1 [\[definitions\]](#) is probably the wrong place for the core-language definition of "relocation operation"

relocation operation

the homogeneous binary operation performed by `std::relocate_at`, consisting of a move-construction immediately followed by a destruction of the source object

ISSUE 2 this definition of "relocation operation" is not good

§ 5.2. Algorithm `relocate_at`

Add a new section after [\[uninitialized.move\]](#):

```
template<class T>
T *relocate_at(T* source, T* dest);

namespace ranges {
    template<Relocatable T>
    T *relocate_at(T* source, T* dest);
}
```

Mandates: `T` shall be a complete non-array object type.

Effects: Equivalent to:

```
uninitialized_move(dest, dest + 1, source);
destroy_at(source);
return std::launder(dest);
```

except that if `T` is trivially relocatable *[basic.types]*, side effects associated with the relocation of the object's value might not happen.

Note: The "as-if-by-memcpy" codepath hidden inside `relocate_at` cannot be implemented by literally `memcpy` (at least not without undefined behavior). At Kona, EWG discussion of Richard Smith's [\[P0593R3\]](#) suggests to me, and Richard confirmed, that a typical library vendor *could* implement this codepath by a call to a hypothetical `out_of_line_memcpy` function whose implementation is invisible to the compiler's optimizer.

§ 5.3. Algorithm `uninitialized_relocate`

Add a new section after [\[uninitialized.move\]](#):

```
template<class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 uninitialized_relocate(ForwardIterator1 first, ForwardIterator1 last,
                                       ForwardIterator2 result);
```

Effects: Equivalent to:

```
result = uninitialized_move(first, last, result);
destroy(first, last);
return result;
```

except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.

Remarks: If an exception is thrown, some objects in the range `[first, last)` are left in a valid but unspecified state.

Note: We are guided by [\[P0884R0\]](#) to make `uninitialized_relocate` unconditionally `noexcept(false)`. This is consistent with `uninitialized_move` and `destroy_at`, both of which are unconditionally `noexcept(false)`.

Note: This algorithm is deliberately different from `relocate_at`-in-a-loop, so that if some move-construction throws an exception, some elements of `[first, last)` may be left in a moved-from state but will never actually be left in a *destroyed* state. If the vendor knows that `T` is nothrow-move-constructible, then they could use `relocate_at`-in-a-loop to speed up that special case.

§ 5.4. Algorithm `uninitialized_relocate_n`

```
template<class ForwardIterator1, class Size, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
uninitialized_relocate_n(ForwardIterator1 first, Size n, ForwardIterator2 result);
```

Effects: Equivalent to:

```
auto pair = uninitialized_move_n(first, n, result);
destroy_n(first, n);
return pair;
```

except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.

Remarks: If an exception is thrown, some objects in the range `[first, std::next(first, n))` are left in a valid but unspecified state.

§ 5.5. Trivially relocatable type

Add a new section in [\[basic.types\]](#):

A move-constructible, destructible object type `T` is a **trivially relocatable** type if it is:

- a trivially copyable type, or
- an array of trivially relocatable type, or
- a (possibly cv-qualified) class type declared with a `[[trivially_relocatable]]` attribute with value `true`, or
- a (possibly cv-qualified) class type which:
 - has no user-provided move constructors or move assignment operators,
 - has no user-provided copy constructors or copy assignment operators,
 - has no user-provided destructors,
 - either is final, or has a final destructor, or has a non-virtual destructor,
 - has no virtual base classes,
 - has no `mutable` or `volatile` members,
 - all of whose members are either of reference type or of trivially relocatable type, and
 - all of whose base classes are trivially relocatable.

[Note: For a trivially relocatable type, the [relocation operation](#) (such as the relocation operations performed by the library functions `std::swap` and `std::vector::resize`) is tantamount to a simple copy of the underlying bytes. —end note]

[Note: It is intended that most standard library types be trivially relocatable types. —end note]

Note: We could simplify the wording by removing the words "either is final, or has a final destructor, or". However, this would lead to the compiler's failing to identify certain (unrealistic) class types as trivially relocatable, when in fact it has enough information to infer that they *are* trivially relocatable in practice. This would be an inaccurate corner case in an otherwise perfectly accurate feature. I tentatively prefer to optimize for both maximum performance and absence of corner cases, over spec simplicity.

Note: There is no special treatment for volatile subobjects (see [\[Subobjects\]](#) and [\[P1152R0\]](#)) nor for possibly overlapping subobjects (see [\[Subobjects\]](#)).

ISSUE 3 The relevant move constructor, copy constructor, and/or destructor must be public and unambiguous. We imply this via the words "A move-constructible, destructible object type". However, "move-constructible" and "destructible" are library concepts, not core language concepts, so maybe it is inappropriate to use them here.

ISSUE 4 Consider the following test cases—

```
struct A {
    struct MA {
        MA(MA&);
        MA(const MA&) = default;
        MA(MA&&) = default;
    };
    mutable MA ma;
    A(const A&) = default;
};
static_assert(not std::is_trivially_relocatable_v<A>);

struct B {
    struct MB {
        MB(const volatile MB&);
        MB(const MB&) = default;
        MB(MB&&) = default;
    };
    volatile MB mb;
    B(const B&) = default;
};
static_assert(not std::is_trivially_relocatable_v<B>);

struct H {
    H(H&&);
};
struct [[trivially_relocatable]] I {
    I(I&&);
};
template<bool Cond>
struct J : std::conditional_t<Cond, H, I> {
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J<false>>);
static_assert(not std::is_trivially_relocatable_v<J<true>>);
```

We *must* find a rule that makes neither **A** nor **B** trivially relocatable, because the move-construction `A(std::move(a))` invokes user-provided copy constructor `MA(MA&)` and the move-construction `B(std::move(b))` invokes user-provided copy constructor `MB(const volatile MB&)`.

In P1144R2, I tried to find a rule that makes **J** trivially relocatable, because the `J : I` pattern was used to implement "conditionally trivial relocatability" for all allocator-aware containers in my libcpp reference implementation. However, this paper adopts the `[[trivially_relocatable(bool)]]` syntax, which means that we don't need to care about `J : I` anymore. The new way to write **J** is simply

```

template<bool Cond>
struct [[trivially_relocatable(!Cond)]] J {
    J(J&&);
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J<false>>);
static_assert(not std::is_trivially_relocatable_v<J<true>>);

```

§ 5.6. [[trivially_relocatable]] attribute

Add a new section after [\[dcl.attr.nouniqueattr\]](#):

The *attribute-token* `trivially_relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It may be applied to the definition of a class. It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, shall have the form

```
( constant-expression )
```

The *constant-expression* shall be an integral constant expression of type `bool`. If no *attribute-argument-clause* is present, it has the same effect as an *attribute-argument-clause* of `(true)`.

If any definition of a class type has a `trivially_relocatable` attribute with value *V*, then each definition of the same class type shall have a `trivially_relocatable` attribute with value *V*. No diagnostic is required if definitions in different translation units have mismatched `trivially_relocatable` attributes.

If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed.

If a class type is declared with the `trivially_relocatable` attribute, and the program relies on observable side-effects of relocation other than a copy of the underlying bytes, the behavior is undefined.

ISSUE 5 "If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" *encouraging* implementations to diagnose. See [this example](#) where a diagnostic might be unwanted.

§ 5.7. Type traits `is_relocatable` etc.

Add new entries to Table 47 in [\[meta.unary.prop\]](#):

Template	Condition	Preconditions
<code>template<class T> struct is_relocatable;</code>	<code>is_move_constructible_v<T></code> is true and <code>is_destructible_v<T></code> is true	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct is_nothrow_relocatable;</code>	<code>is_relocatable_v<T></code> is true and both the indicated move-constructor and the destructor are known not to throw any exceptions.	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct is_trivially_relocatable;</code>	T is a trivially relocatable type.	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.

§ 5.8. `Relocatable` concept

Add a new section after [\[concept.moveconstructible\]](#):

```
template<class T>  
concept Relocatable = MoveConstructible<T> && Destructible<T>;
```

Note: This concept is exactly equivalent to `MoveConstructible<T>`.

Note: My understanding is that there are no standard concepts with `Trivially` in the name. Therefore I do not propose to add a concept named `TriviallyRelocatable`.

§ 6. Rationale and alternatives

§ 6.1. Why not destructive move?

[As discussed in EWGI at San Diego](#), this proposal does *not* give us a general user-provided "destructive move" facility.

- Denis Bider's [\[P0023R0\]](#) and Pablo Halpern's [\[N4158\]](#) went in that direction and did not succeed. People have been chasing "destructive move" for decades; maybe it's time to try something different.
- We get the performance benefit only when the library (e.g. `std::vector::resize`) can detect that "relocate/destructive move" is tantamount to memcopy. If we permit a user-provided "destructive move" operation, we must also design a way for the user to warrant that their "destructive move" is tantamount to memcopy. No previous proposal has shown how to do this.
- P1144's approach is explicitly based on existing industry practice: [\[Folly\]](#), [\[EASTL\]](#), and [\[BSL\]](#) all use this exact idea in practice and it seems to work for them. Marc Glisse has been integrating the same idea into GNU libstdc++; see [\[Deque\]](#). The term "relocation" is due to [\[EASTL\]](#) (`has_trivial_relocate`) and [\[Folly\]](#) (`IsRelocatable`). The same concept appears in pre-C++11 libraries under the name "movable": Qt (`Q_MOVABLE_TYPE`) and [\[BSL\]](#) (`IsBitwiseMoveable`). P1144's sole innovation is to give a consistent set of core-language rules by which the compiler can deduce the trivial relocatability of some class types which follow the Rule of Zero.

§ 6.2. Attribute `[[maybe_trivially_relocatable]]`

The Clang patch currently available on Godbolt Compiler Explorer supports both `[[clang::trivially_relocatable]]` and another attribute called `[[clang::maybe_trivially_relocatable]]`, which John McCall requested that I explore.

P1144 talks in terms of "warranting" that a class is trivially relocatable; this can be done manually via annotation, or as a sort of "auto-warrant" done automatically by the compiler when we follow the Rule of Zero.

But another way of looking at it is that trivial relocatability is always inherited in a straightforward way, and when we use the attribute, we are "overruling" a decision made by the compiler — a decision it made for maybe several different reasons. So we can have two different levels of "overruling":

- The first level, `[[clang::maybe_trivially_relocatable]]`, means "I warrant that even though I may have user-provided, non-defaulted, special member functions, I have designed them so that my relocation operation will not do anything substantially different from memberwise relocation." So if all of my member and base subobjects are trivially relocatable (and not mutable and not volatile), then I myself will be trivially relocatable.
- The second level, `[[clang::trivially_relocatable]]`, means "I warrant that even though I may have user-provided, non-defaulted, special member functions, and even though I may have non-trivially relocatable (or mutable or volatile) subobjects, I have designed them so that my

relocation operation will not do anything substantially different from `memcpy`." So I myself will be trivially relocatable *no matter what my subobjects claim about themselves*.

Using `[[clang::trivially_relocatable]]`, I can write a class that "overrides" a decision made by one of its members. I can make a trivially relocatable class that contains a data member of type `boost::interprocess::offset_ptr<int>` (using fundamentally non-trivial pieces to build a trivially relocatable whole: see [B](#) below). I can make a trivially relocatable class that contains a data member of type `boost::shared_ptr<int>` (wrapping a fundamentally trivial type in a wrapper that explicitly warrants that triviality in a compiler-visible way: see [C](#) below).

Using `[[clang::maybe_trivially_relocatable]]`, I am forbidden to write either of those classes; but I also am forbidden to write a trivially relocatable class that contains a data member of type `std::list<int>` (wrapping a fundamentally non-trivial type in a wrapper that explicitly, *incorrectly*, warrants its own triviality: see [D](#) below).

The single "Strongly Against" vote on [poll 1](#) was due to concerns that P1144 permits a class to warrant its own trivial relocatability — overruling the compiler's assumptions — not only when the compiler's assumptions are based on the presence of special members ([A](#) below), but also when the compiler's assumptions are based partly or wholly on the non-triviality of member or base subobjects ([B](#), [C](#), [D](#) below).

I see cases [B](#) and [C](#) as a "feature, not a bug"; but both Ed Catmur (in polling) and John McCall (in Clang review) have indicated that they would prefer to allow case [A](#) but forbid cases [B](#) and [C](#) (and of course [D](#)) by standardizing the equivalent of `[[clang::maybe_trivially_relocatable]]` in lieu of `[[clang::trivially_relocatable]]`. ([Here is the example on Compiler Explorer.](#))

```

// Everyone agrees that we want to allow this.
struct [[trivially_relocatable]] A {
    int *p;
    int i;
    static int g;
    A() : p(&g), i(0) {}
    A(A&& rhs) : p(&g), i(rhs.i) {}
    A& operator=(A&& rhs) { i = rhs.i; return *this; }
};

// P1144 explicitly wants to allow this; but some want to forbid it.
struct [[trivially_relocatable]] B {
    boost::interprocess::offset_ptr<int> p;
    int i;
    B() : p(&i), i(0) {}
    B(B&& rhs) : p(&i), i(rhs.i) {}
    B& operator=(B&& rhs) { i = rhs.i; return *this; }
};

// Arguably this code has poor style.
// P1144 explicitly wants to allow this; but some want to forbid it.
struct [[trivially_relocatable]] C {
    boost::shared_ptr<int> p;
    C(C&&) = default;
    C& operator=(C&&) = default;
};

// This code is buggy.
// P1144 allows you to shoot yourself in the foot like this.
struct [[trivially_relocatable]] D {
    std::list<int> lst;
    D(D&&) = default;
    D& operator=(D&&) = default;
};

```

I believe a major problem with `[[clang::maybe_trivially_relocatable]]` is that it requires us to talk about class types which are "not impediments to trivial relocatability" yet are not themselves trivially relocatable (because they are not relocatable at all). For example, libc++'s `optional` has a base class which is destructible but not move-constructible:

```

template<class T>
struct optional_destruct_base {
    bool engaged;
    union {
        char dummy;
        T t;
    };
    ~optional_destruct_base() { if (engaged) t.~T(); }
};

template<class T>
struct optional_move_base : optional_destruct_base<T> { ... };

template<class T>
class optional : optional_move_base<T> { ... };

```

In a P1144 `[[trivially_relocatable]]` world, this is no problem; the annotation happens at a very high level, where the class designer of `optional` can be sure that their move-constructor (inherited from `optional_move_base`) and their destructor (inherited from `optional_destruct_base`) will play together in the right way.

In a `[[clang::maybe_trivially_relocatable]]` world, someone needs to annotate `optional_destruct_base` so that its non-relocatability will not block the high-level `optional` from becoming trivially relocatable. But then we have a weird scenario: the non-moveable class `optional_destruct_base` seems to be claiming that it is "trivially relocatable," even though it is not relocatable, nor even moveable! Yet, if we don't annotate it, we break the high-level `optional`. This strikes me as a spooky situation where actions in one part of the codebase can have effects on distant parts.

Lastly, the concern over `[[trivially_relocatable]]` seems to be based on the assumption that working programmers will frequently use this attribute in practice. In reality, I don't think they will. P1144 is designed so that the Rule of Zero will do the right thing, and all library types will do the right thing. Most programmers won't ever use the annotation, just as most programmers don't use `[[maybe_unused]]` or `[[no_unique_address]]`.

The proposed attribute is a simple, sharp knife. It cuts what you point it at; point it carefully. `[[maybe_trivially_relocatable]]` is a slightly dull knife; it doesn't necessarily cut on the first try. A dull knife is often more dangerous than a sharp one.

§ 6.3. Should `relocate_at` be a customization point?

No. See [P1144R2 section 5.4](#) for discussion of this approach, which was taken by Pablo Halpern's [\[N4158\]](#). See also "[Trivially Relocatable versus Destructive Movable](#)" (2018-09-28).

N4158's customization-point approach seems to have a very high cost-to-benefit ratio. I am satisfied with P1144's avoiding that approach.

§ 6.4. Unintuitive `is_nothrow_relocatable`

Consider a type such as

```
struct [[trivially_relocatable]] Widget {
    int i;
    Widget(Widget&&) : i(rhs.i) {}
};
static_assert(not std::is_nothrow_move_constructible_v<Widget>);
static_assert(not std::is_nothrow_relocatable_v<Widget>);
static_assert(std::is_trivially_relocatable_v<Widget>);
```

Since `Widget` is non-nothrow move-constructible, P1144 calls it non-nothrow relocatable. So, looking at how `Widget` interacts with the type-traits, we are in the awkward position that `Widget` simultaneously claims "My relocation operation might throw" and "My relocation operation is trivial." These claims seem inconsistent.

This is a real-world concern because GNU libstdc++'s `std::deque<T, std::allocator<T>>` works like `Widget`: its move-constructor is `noexcept(false)` (it must allocate) but it is trivially relocatable. As of 2019-01-18, libstdc++ marks its `deque` as trivially relocatable (see [\[Deque\]](#)).

However, I believe that it would be incorrect and unsafe for the library to claim that `Widget` was "nothrow relocatable." "Nothrow relocatable" should imply that a generic algorithm could relocate it (as if by `std::uninitialized_relocate`) without worrying about catching exceptions. "`T` is trivially relocatable" means that `T` is relocatable *as if* by `memcpy`; it does not mean that every relocation of `T` *must* be performed *literally* by `memcpy`.

I believe P1144's proposed behavior is the best behavior. However, another plausible behavior would be to simply eliminate the `is_nothrow_relocatable` type-trait from the standard library. If we don't provide `is_nothrow_relocatable`, then we don't have to defend its mildly unintuitive behavior.

§ 7. Acknowledgements

Thanks to Elias Kosunen, Niall Douglas, and John Bandela for their feedback on early drafts of this paper.

Many thanks to Matt Godbolt for allowing me to install the prototype Clang implementation on Compiler Explorer (godbolt.org). See also [\[Announcing\]](#).

Thanks to Nicolas Lesser for his relentless feedback on drafts of P1144R0, and for his helpful review comments on the Clang implementation [\[D50119\]](#).

Thanks to Howard Hinnant for appearing with me on [\[CppChat\]](#), and to Jon Kalb and Phil Nash for hosting us.

Thanks to Pablo Halpern for [\[N4158\]](#), to which this paper bears a striking and coincidental resemblance—including the meaning assigned to the word "trivial," and the library-algorithm approach to avoiding the problems with "lame duck objects" discussed in [the final section](#) of [\[N1377\]](#). See [discussion of N4034 at Rapperswil](#) (June 2014) and [discussion of N4158 at Urbana](#) (November 2014).

Significantly different approaches to this problem have previously appeared in Rodrigo Castro Campos's [\[N2754\]](#), Denis Bider's [\[P0023R0\]](#) (introducing a core-language "relocation" operator), and Niall Douglas's [\[P1029R1\]](#) (treating relocatability as an aspect of move-construction in isolation, rather than an aspect of the class type as a whole).

Thanks to John McCall for his thought-provoking review comments on the Clang implementation [\[D50119\]](#).

Thanks to Marc Glisse for his work integrating a "trivially relocatable" trait into GNU libstdc++ and for answering my questions on [GCC bug 87106](#).

Thanks to Jens Maurer for his feedback on this paper at Kona 2019.

§ Appendix A: Straw polls

§ Polls taken of the Internet between 2018-11-12 and 2018-11-21

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>We approve of the general idea that user-defined classes should be able to</i>	<i>6</i>	<i>1</i>	<i>0</i>	<i>0</i>	<i>1</i>

warrant their own trivial relocatability via a standard mechanism.					
We approve of the general idea that user-defined classes which follow the Rule of Zero should inherit the trivial relocatability of their bases and members.	7	1	0	0	0
Nobody should be able to warrant the trivial relocatability of <code>class C</code> except for <code>class C</code> itself (i.e., we do not want to see a customization point analogous to <code>std::hash</code>).	4	2	2	0	0
A class should be able to warrant its own trivial relocatability via the attribute <code>[[trivially_relocatable]]</code> , as proposed in this paper (P1144R0).	3	0	3	1	0
A class should be able to warrant its own trivial relocatability via some attribute, but not necessarily under that exact name.	2	0	4	1	0
A class should be able to warrant its own trivial relocatability as proposed in this paper (P1144R0), but via a contextual keyword rather than an attribute.	0	2	3	3	0
If a trait with the semantics of <code>is_trivially_relocatable<T></code> is added to the <code><type_traits></code> header, the programmer should be permitted to specialize it for program-defined types (i.e., we want to see that trait itself become a customization point analogous to <code>std::hash</code>).	0	1	0	1	5
Trivial relocatability should be assumed by default. Classes such as those in Appendix C should indicate their non-trivial relocatability via an opt-in mechanism.	0	0	0	3	5
To simplify Conditionally trivial relocation , if an attribute with the semantics of <code>[[trivially_relocatable]]</code> is added, it should take a boolean argument.	1	1	3	2	0
The algorithm <code>uninitialized_relocate(first, last, d_first)</code> should be added to the <code><memory></code> header, as proposed in this paper (P1144R0).	0	4	1	1	0
The type trait <code>is_relocatable<T></code> (and its <code>_v</code> version) should be added to the <code><type_traits></code> header, as proposed in this paper (P1144R0).	0	2	3	0	1
If <code>is_relocatable<T></code> is added, then we should also add <code>is_nothrow_relocatable<T></code> (and its <code>_v</code> version), as proposed in this paper (P1144R0).	1	4	2	0	0
The type trait <code>is_trivially_relocatable<T></code> (and its <code>_v</code> version) should be added to the <code><type_traits></code> header, under that exact name, as proposed in	3	3	1	0	0

this paper (P1144R0).					
We approve of a trait with the semantics of <code>is_trivially_relocatable<T></code> , but not necessarily under that exact name. (For example, <code>is_bitwise_relocatable</code> .)	3	3	0	1	0
If <code>is_trivially_relocatable<T></code> is added, under that exact name, then the type trait <code>is_trivially_swappable<T></code> (and its <code>_v</code> version) should also be added to the <code><type_traits></code> header.	0	3	3	0	0

The "Strongly Against" vote on poll 1 was due to concerns that P1144 permits a class to warrant its own trivial relocatability, overruling the compiler's assumptions, not only when the compiler's assumptions are based on the presence of special members, but also when the compiler's assumptions are based partly or wholly on the non-triviality of member or base subobjects. See further discussion under [§ 6.2 Attribute `\[\[maybe\_trivially\_relocatable\]\]`](#).

The "Against" vote on poll 10, `uninitialized_relocate`, was due to its exception guarantee, which was weaker in P1144R0. P1144R1 has strengthened the guarantee (and tightened the constraint on the source iterator from `InputIterator` to `ForwardIterator`) to better match the other `uninitialized_foo` algorithms.

The "Strongly Against" vote on poll 11, `is_relocatable`, was from a desire to save the name `relocatable` for something different, such as a built-in destructive-move operation.

§ Poll taken of EWGI at San Diego on 2018-11-07

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
Should we commit additional committee time to solving the problem P1144R0 is trying to solve, knowing it will leave less time to other work?	8	3	0	0	0

§ Polls taken of SG14 at CppCon on 2018-09-26

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
The type trait <code>is_trivially_relocatable<T></code> (and its <code>_v</code> version) should be added to the <code><type_traits></code> header, under that exact name, as proposed	1	20	7	1	0

in this paper.					
We approve of a trait with the semantics of <i>is_trivially_relocatable</i> <T>, but not necessarily under that exact name. (For example, <i>is_bitwise_relocatable</i> .)					
We approve of the general idea that user-defined classes should be able to warrant their own trivial relocatability.					
	15	12	1	0	0
	25	5	2	0	0

§ Appendix B: Sample code

§ Reference implementation of `relocate_at` and `uninitialized_relocate`

```

template<class T>
T *relocate_at(T *source, T *dest)
{
    if constexpr (std::is_trivially_relocatable_v<T>) {
        std::memmove(dest, source, sizeof(T));
        return std::launder(dest);
    } else {
        T *result = ::new (dest) (std::move(*source));
        source->~T();
        return result;
    }
}

template<class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 uninitialized_relocate(ForwardIterator1 first, ForwardIterator1 last,
                                       ForwardIterator2 result)
{
    using T = typename iterator_traits<ForwardIterator2>::value_type;
    using U = decltype(std::move(*first));
    constexpr bool memcopyable = (std::is_same_v<T, std::remove_ref_t<U>> && std::is_trivially_relocatable_v<T>);
    constexpr bool both_contiguous = (std::is_pointer_v<ForwardIterator1> && std::is_pointer_v<ForwardIterator2>);
    constexpr bool nothrow_relocatable = std::is_nothrow_constructible_v<T, U>;

    if constexpr (memcopyable && both_contiguous) {
        std::size_t nbytes = (char *)last - (char *)first;
        if (nbytes != 0) {
            std::memmove(std::addressof(*result), std::addressof(*first), nbytes);
            result += (last - first);
        }
    } else if constexpr (nothrow_relocatable) {
        for (; first != last; (void)++result, ++first) {
            ::new (static_cast<void*>(std::addressof(*result))) T(std::move(*first));
            std::destroy_at(std::addressof(*first));
        }
    } else {
        result = std::uninitialized_move(first, last, result);
        std::destroy(first, last);
    }
    return result;
}

```

§ Conditionally trivial relocation

We expect, but do not require, that `std::optional<T>` should be trivially relocatable if and only if `T` itself is trivially relocatable.

The following abbreviated implementation shows how to achieve an `optional<T>` which has the same trivial-move-constructibility as `T`, the same trivial-destructibility as `T`, and the same trivial-relocatability as `T`.

The primitives of move-construction and destruction are provided by four specializations of `optional_a`; then the public `optional` extends the appropriate specialization of `optional_a` and applies a conditional `[[trivially_relocatable]]` attribute.

```
template<class T>
class [[trivially_relocatable(is_trivially_relocatable_v<T>)]] optional : optional_base<T>
{
    using optional_base<T>::optional_base;
};

template<class T, bool D = is_trivially_destructible_v<T>, bool M = is_trivially_move_constructible_v<T>
class optional_base {
    // NOT SHOWN
};
```

I have implemented the entire Standard Library using the proposed `[[trivially_relocatable]]` attribute; you can find the source code [on my GitHub](#) and explore the resulting codegen [on Godbolt Compiler Explorer](#).

I have also implemented case studies for `std::optional` and `std::deque`, in each of the alternative styles:

Style	Size of <code>optional</code> diff (lines)	Size of <code>deque</code> diff (lines)
<code>[[trivially_relocatable]]</code>	<u>-2, +14</u>	<u>-18, +52</u>
<code>[[maybe_trivially_relocatable]]</code>	<u>problematic</u>	<u>-5, +5</u>
<code>[[trivially_relocatable(bool)]]</code>	<u>-1, +1</u>	<u>-1, +17</u>

For why one entry in this table is "problematic," see [§ 6.2 Attribute `\[\[maybe\_trivially\_relocatable\]\]`](#).

§ Appendix C: Examples of non-trivially relocatable class types

§ Class contains pointer to self

This fictional `short_string` illustrates a mechanism that can apply to any small-buffer-optimized class. `libc++'s std::function` uses this mechanism (on a 24-byte buffer) and is thus not trivially relocatable.

However, different mechanisms for small-buffer optimization exist. `libc++'s std::any` also achieves small-buffer optimization on a 24-byte buffer, without (necessarily) sacrificing trivial relocatability.

```
struct short_string {
    char *data_ = buffer_;
    size_t size_ = 0;
    char buffer_[8] = {};

    const char *data() const { return data_; }

    short_string() = default;
    short_string(const char *s) : size_(strlen(s)) {
        if (size_ < sizeof buffer_)
            strcpy(buffer_, s);
        else
            data_ = strdup(s);
    }
    short_string(short_string&& s) {
        memcpy(this, &s, sizeof(*this));
        if (s.data_ == s.buffer_)
            data_ = buffer_;
        else
            s.data_ = nullptr;
    }
    ~short_string() {
        if (data_ != buffer_)
            free(data_);
    }
};
```

§ Allocated memory contains pointer to self

`std::list` needs somewhere to store its "past-the-end" node, commonly referred to as the "sentinel node," whose `prev` pointer points to the list's last node. If the sentinel node is allocated on the heap, then `std::list` can be trivially relocatable; but if the sentinel node is placed within the `list` object itself (as happens on `libc++` and `libstdc++`), then relocating the `list` object requires fixing up the

list's last node's `next` pointer so that it points to the new sentinel node inside the destination `list` object. This fixup of an arbitrary heap object cannot be simulated by `memcpy`.

Traditional implementations of `std::set` and `std::map` also store a "past-the-end" node inside themselves and thus also fall into this category.

```
struct node {
    node *prev_ = nullptr;
    node *next_ = nullptr;
};

struct list {
    node n_;
    iterator begin() { return iterator(n_.next_); }
    iterator end() { return iterator(&n_); }
    list(list&& l) {
        if (l.n_.next_) l.n_.next_->prev_ = &n_; // fixup
        if (l.n_.prev_) l.n_.prev_->next_ = &n_; // fixup
        n_ = l.n_;
        l.n_ = node{};
    }
    // ...
};
```

§ Class invariant depends on `this`

The `offset_ptr` provided by [\[Boost.Interprocess\]](#) is an example of this category.

```
struct offset_ptr {
    uintptr_t value_;

    uintptr_t here() const { return uintptr_t(this); }
    uintptr_t distance_to(void *p) const { return uintptr_t(p) - here(); }
    void *get() const { return (void*)(here() + value_); }

    offset_ptr() : value_(distance_to(nullptr)) {}
    offset_ptr(void *p) : value_(distance_to(p)) {}
    offset_ptr(const offset_ptr& rhs) : value_(distance_to(rhs.get())) {}
    offset_ptr& operator=(const offset_ptr& rhs) {
        value_ = distance_to(rhs.get());
        return *this;
    }
    ~offset_ptr() = default;
};
```


§ Program invariant depends on `this`

In the following snippet, `struct Widget` is relocatable, but not trivially relocatable, because the relocation operation of destroying a `Widget` at point A and constructing a new `Widget` at point B has behavior that is observably different from a simple `memcpy`.

```
std::set<void*> registry;

struct registered_object {
    registered_object() { registry.insert(this); }
    registered_object(registered_object&&) = default;
    registered_object(const registered_object&) = default;
    registered_object& operator=(registered_object&&) = default;
    registered_object& operator=(const registered_object&) = default;
    ~registered_object() { registry.erase(this); }
};

struct Widget : registered_object {};
```

§ Appendix D: Implementation experience

A prototype Clang/libc++ implementation is at

- github.com/Quuxplusone/clang/tree/trivially-relocatable
- github.com/Quuxplusone/libcxx/tree/trivially-relocatable
- godbolt.org, under the name "x86-64 clang (experimental P1144)"
- reviews.llvm.org/D50119

Side-by-side case studies of `[[trivially_relocatable]]`, `[[maybe_trivially_relocatable]]`, and `[[trivially_relocatable(bool)]]` are given in [Conditionally trivial relocation](#).

As of November 2018, libstdc++ performs the `vector::resize` optimization for any type which has manually specialized `std::__is_trivially_relocatable`. (See it on Compiler Explorer [here](#).) Manual specialization is also the approach used by [Folly](#), [EASTL](#), and [BSL](#). As of 2019-01-18, the only libstdc++ library type for which `__is_trivially_relocatable` has been specialized is `deque`; see [Deque](#) and [§ 6.4 Unintuitive is_nothrow_relocatable](#).

§ Index

§ Terms defined by this specification

relocation operation, in §5.1

trivially relocatable, in §5.5

§ References

§ Normative References

[N4800]

Richard Smith. Working Draft, Standard for Programming Language C++. 21 January 2019.
URL: <https://wg21.link/n4800>

§ Informative References

[Announcing]

Arthur O'Dwyer. Announcing "trivially relocatable". July 2018. URL:
<https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/>

[Bench]

Arthur O'Dwyer. Benchmark code from "The Best Type Traits C++ Doesn't Have". April 2018.
URL: <https://github.com/Quuxplusone/from-scratch/blob/095b246d/cppnow2018/benchmark-relocatable.cc>

[Boost.Interprocess]

Ion Gaztañaga. Mapping Address Independent Pointer: offset_ptr. 2005. URL:
https://www.boost.org/doc/libs/1_67_0/doc/html/interprocess/offset_ptr.html

[BSL]

Bloomberg. bslmf::IsBitwiseMoveable: bitwise moveable trait metafunction. 2013–2017. URL:
https://github.com/bloomberg/bde/blob/master/groups/bsl/bslmf/bslmf_isbitwisemoveable.h#L17

[Contra]

Arthur O'Dwyer. Contra built-in library types. April 2018. URL:
<https://quuxplusone.github.io/blog/2018/04/15/built-in-library-types/>

[CppChat]

Howard Hinnant; Arthur O'Dwyer. cpp.chat episode 40: It works but it's undefined behavior.
August 2018. URL: <https://www.youtube.com/watch?v=8u5Qi4FgTP8>

[D50119]

Arthur O'Dwyer; Nicolas Lesser; John McCall. [Compiler support for P1144R0 __is_trivially_relocatable\(T\)](#). July 2018. URL: <https://reviews.llvm.org/D50119>

[Deque]

Marc Glisse. [Improve relocation ... \(__is_trivially_relocatable\): Specialize for deque](#). November 2018. URL: <https://github.com/gcc-mirror/gcc/commit/a9b9381580de611126c9888c1a6c12a77d9b682e>

[EASTL]

Paul Pedriana. [N2271: EASTL — Electronic Arts Standard Template Library](#). URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2271.html#type_traits_extensions

[FixedCapacityVector]

Arthur O'Dwyer. [P1144 case study: Moving a `fixed\\_capacity\\_vector`](#). URL: <https://quuxplusone.github.io/blog/2019/02/22/p1144-fixed-capacity-vector/>

[Folly]

Facebook. [Folly documentation on "Object Relocation"](#). URL: <https://github.com/facebook/folly/blob/master/folly/docs/FBVector.md#object-relocation>

[FollyIssue889]

Arthur O'Dwyer. [Traits.h marks std::list as trivially relocatable, but in fact it is not](#). URL: <https://github.com/facebook/folly/issues/889>

[LibcxxAny]

Eric Fiselier. [libc++ implementation of std::any \(trivially relocatable\)](#). July 2016. URL: <https://github.com/llvm-mirror/libcxx/blob/8fdc4918/include/any#L389-L394>

[LibcxxFunction]

Howard Hinnant et al. [libc++ implementation of std::function \(non-trivially relocatable\)](#). URL: <https://github.com/llvm-mirror/libcxx/blob/4e7ffcaa/include/functional#L1719-L1734>

[LibstdcxxFunction]

Doug Gregor et al. [libstdc++ implementation of std::function \(trivially relocatable\)](#). URL: https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/std_function.h

[N1377]

Howard Hinnant; Peter Dimov; Dave Abrahams. [N1377: A Proposal to Add Move Semantics Support to the C++ Language](#). September 2002. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm>

[N2754]

Rodrigo Castro Campos. [N2754: TriviallyDestructibleAfterMove and TriviallyReallocatable \(rev 3\)](#). September 2008. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html>

[N4158]

Pablo Halpern. [N4158: Destructive Move \(rev 1\)](#). October 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf>

[P0023R0]

Denis Bider. [P0023R0: Relocator: Efficiently Moving Objects](http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf). April 2016. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf>

[P0593R3]

Richard Smith. [P0593R3: Implicit creation of objects for low-level object manipulation](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0593r3.html). January 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0593r3.html>

[P0884R0]

Nicolai Josuttis. [P0884R0: Extending the noexcept Policy, Rev. 0](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0884r0.pdf). February 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0884r0.pdf>

[P1007R3]

Timur Doumler; Chandler Carruth. [P1007R3: std::assume_aligned](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1007r3.pdf). November 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1007r3.pdf>

[P1029R1]

Niall Douglas. [P1029R1: SG14 \[\[move_relocates\]\]](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1029r1.pdf). August 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1029r1.pdf>

[P1152R0]

JF Bastien. [P1152R0: Deprecating volatile](http://wg21.link/p1152r0). July 2018. URL: <http://wg21.link/p1152r0>

[Subobjects]

Arthur O'Dwyer. [When is a trivially copyable object not trivially copyable?](https://quuxplusone.github.io/blog/2018/07/13/trivially-copyable-corner-cases/). July 2018. URL: <https://quuxplusone.github.io/blog/2018/07/13/trivially-copyable-corner-cases/>

§ Issues Index

ISSUE 1 [definitions] is probably the wrong place for the core-language definition of "relocation operation" [↵](#)

ISSUE 2 this definition of "relocation operation" is not good [↵](#)

ISSUE 3 The relevant move constructor, copy constructor, and/or destructor must be public and unambiguous. We imply this via the words "A move-constructible, destructible object type". However, "move-constructible" and "destructible" are library concepts, not core language concepts, so maybe it is inappropriate to use them here. [↵](#)

ISSUE 4 Consider the following test cases—

```
struct A {
    struct MA {
        MA(MA&);
        MA(const MA&) = default;
        MA(MA&&) = default;
    };
    mutable MA ma;
    A(const A&) = default;
};
static_assert(not std::is_trivially_relocatable_v<A>);

struct B {
    struct MB {
        MB(const volatile MB&);
        MB(const MB&) = default;
        MB(MB&&) = default;
    };
    volatile MB mb;
    B(const B&) = default;
};
static_assert(not std::is_trivially_relocatable_v<B>);

struct H {
    H(H&&);
};
struct [[trivially_relocatable]] I {
    I(I&&);
};
template<bool Cond>
struct J : std::conditional_t<Cond, H, I> {
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J<false>>);
static_assert(not std::is_trivially_relocatable_v<J<true>>);
```

We *must* find a rule that makes neither **A** nor **B** trivially relocatable, because the move-construction `A(std::move(a))` invokes user-provided copy constructor `MA(MA&)` and the move-construction `B(std::move(b))` invokes user-provided copy constructor `MB(const volatile MB&)`.

In P1144R2, I tried to find a rule that makes **J** trivially relocatable, because the `J : I` pattern was used to implement "conditionally trivial relocatability" for all allocator-aware containers in my libcpp reference implementation. However, this paper adopts the `[[trivially_relocatable(bool)]]` syntax, which means that we don't need to care about `J : I` anymore. The new way to write **J** is simply

```
template<bool Cond>
struct [[trivially_relocatable(!Cond)]] J {
    J(J&&);
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J<false>>);
static_assert(not std::is_trivially_relocatable_v<J<true>>);
```

↩

ISSUE 5 "If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" *encouraging* implementations to diagnose. See [this example](#) where a diagnostic might be unwanted. ↩